

URL SHORTENER

Python-Based Web Application for Creating and Managing Short URLs

Student Name	SANTHOSH JAMPALA
Internship Domain	Python / Python for Data Science
Project	URL Shortener
Technologies	Python, Flask, SQLite, HTML, CSS, JavaScript
Repository	upskillcampus

1. Project Overview

The URL Shortener is a Python-based web application developed to convert long URLs into shorter, easy-to-share links. The application validates the submitted URL, generates a unique short code, stores the URL mapping in SQLite, and redirects users from the shortened URL to the original destination.

2. Problem Statement

Long URLs can be difficult to share and manage. A URL shortening system provides compact links while maintaining a mapping between the shortened URL and its original URL. This project was developed to gain practical experience in Python web development, routing, database connectivity, validation, and application data management.

3. Objectives

- Develop a functional URL shortening web application using Python and Flask.
- Generate unique short codes for submitted URLs.
- Store original URLs, short codes, and click information using SQLite.
- Validate submitted URLs and handle invalid input.
- Detect duplicate URLs and reuse an existing short code.
- Provide URL history and basic URL management features.
- Practice backend development, database operations, frontend integration, testing, and debugging.

4. Technologies Used

Python: Core programming language and application logic.

Flask: Web framework for routing, request handling, and page rendering.

SQLite: Local database for URL mappings and click counts.

HTML/CSS: User interface structure and styling.

JavaScript: Client-side copy and search interactions.

5. Key Features

- **URL Validation:** Checks submitted HTTP/HTTPS URLs.
- **Short-Code Generation:** Creates compact alphanumeric codes.
- **Duplicate URL Handling:** Returns the existing short URL when the same original URL is submitted again.
- **Redirection:** Opens the original URL through the short code.
- **Click Tracking:** Records the number of visits to each shortened URL.
- **URL History:** Displays stored URLs and click counts.
- **Search/Filter:** Filters URL history.
- **Copy:** Copies a generated short URL.
- **Delete:** Removes a stored shortened URL.

6. System Workflow

1. User enters a long URL.
2. Flask receives the request and validates the URL.
3. SQLite is checked for an existing URL.
4. If it exists, the existing short code is returned.
5. Otherwise, a unique short code is generated and stored.
6. The shortened URL is displayed to the user.
7. When opened, the short code is used to retrieve the original URL and increment the click count.

7. Database Design

The application uses an SQLite database named **urls.db**. The main URL table contains:

- **id:** Primary key and record identifier.
- **original_url:** Original URL submitted by the user.
- **short_code:** Unique code used in the shortened URL.
- **clicks:** Number of times the shortened URL has been opened.

8. Project Structure

URL_Shortener/

```
■■■ app.py
■■■ urls.db
■■■ requirements.txt
■■■ README.md
■■■ templates/
■ ■■■ index.html
■ ■■■ history.html
■■■ static/
    ■■■ style.css
```

9. Testing Performed

- Valid HTTP/HTTPS URL → Short URL generated.
- Invalid URL → Validation error displayed.
- Duplicate URL → Existing short URL returned.
- Short URL opened → Redirected to original URL.
- Short URL opened repeatedly → Click count increased.
- History page → Stored URLs and click counts displayed.
- Search → Matching history records filtered.
- Copy → Short URL copied to clipboard.
- Delete → Selected URL removed from history.

10. Learning Outcomes

- Gained practical experience with Python and Flask web application development.
- Learned how backend routes communicate with HTML forms and browser requests.
- Practiced SQLite database creation and CRUD operations.
- Improved understanding of validation, duplicate handling, redirection, and click tracking.
- Practiced debugging, testing, code organization, and documentation.

11. GitHub Repository Links

Repository: <https://github.com/jsanthosh20/upskillcampus>

Project Code: <https://github.com/jsanthosh20/upskillcampus/blob/main/app.py>

Project Report: https://github.com/jsanthosh20/upskillcampus/blob/main/URLShortener_SANTHOSH_USC_UCT.pdf

12. Conclusion

The URL Shortener project provided practical experience in Python, Flask, SQLite, frontend development, routing, database management, validation, testing, and debugging. It demonstrates how Python can be used to build a complete web application with persistent data storage and user-facing functionality.